

National Institute of Technology Karnataka Surathkal
Department of Information Technology



IT 252 DATABASE SYSTEMS

**Basics of Functional Dependencies and
Normalization for Relational Databases**

Dr. Jayashree T R

Course Outline

Course Plan: Theory:

Part A: Parallel Computer Architectures

Week 1: Introduction to the course, highlighting the data and databases, and related basic concepts. Advantages and need for a database system.

Week 2: Attributes, tuples, relational schema, conceptual model, introduction to SQL-DML, DDL, creating relations/ tables, access and manipulate data.

Week 3,4: Data model, relational model, and SQL, relational data model and relational database constraints, SQL data definition and data types, specifying constraints, basic retrieval queries, complex queries, triggers, views, schema modification.

Week 5-6: ER model development, entity types/sets, attribute, relationship types/sets, simple employee database conceptual design using ER concepts, ER to relational mapping algorithm, EER model.

Course Outline

Week 7,8 : Functional dependency definition, need of normalization, anomalies and redundant information in tuples, normalization steps (1NF, 2NF, 3NF), BCNF, introduction to the higher normal forms.

Week 9: Procedural query languages: relational algebra and SQL-SELECT, PROJECT, RENAME, binary and JOIN operations with examples.

Non-Procedural query languages: relational calculus, and SQL-tuple relational calculus, existential and universal quantifier with examples.

Week 10,11 : Transaction management, schedule and serializability, concurrency control, 2-phase lock, recovery mechanism: undo/redo values.

Week 12,13: Disk storage- basic file structures, ordered/unordered. binary search, hashing, indexing, importance of indexes, primary and secondary indexing methods, clustered, B tree, B++ trees.

Week 14: Current trends in database system, introduction to data warehousing, data mining.

Practical: MySQL commands

Project: Team of 2 or 3 members

1. Informal Design Guidelines for Relational Databases

- Relational database design ultimately produces a set of relations. The implicit goals of the design activity are **information preservation** and **minimum redundancy**.
- **Information preservation in terms of** maintaining all concepts, including attribute types, entity types, and relationship types as well as **generalization/specialization relationships**, which are described using a model such as the EER model.
- **Minimizing redundancy implies** minimizing redundant storage of the same information and reducing the need for multiple updates to maintain consistency across multiple copies of the same information in response to real-world events that require making an update.

1. Informal Design Guidelines for Relational Databases

- What is relational database design?
 - The grouping of attributes to form "good" relation schemas
- Two levels of relation schemas
 - The logical "user view" level
 - The storage "base relation" level
- Design is concerned mainly with base relations
- What are the criteria for "good" base relations?

1. Informal Design Guidelines for Relational Databases

Four informal guidelines that may be used as measures to determine the **quality of relation schema design**:

- Making sure that the semantics of the attributes is clear in the schema
- Reducing the redundant information in tuples
- Reducing the NULL values in tuples
- Disallowing the possibility of generating *spurious tuples*

These measures are not always independent of one another, as we will see.

1.1 Semantics of the Relational

Attributes must be clear

- GUIDELINE 1: Informally, each tuple in a relation should represent one entity or relationship instance. (Applies to individual relations and their attributes).
 - Attributes of different entities (EMPLOYEEs, DEPARTMENTs, PROJECTs) should not be mixed in the same relation
 - Only foreign keys should be used to refer to other entities
 - Entity and relationship attributes should be kept apart as much as possible.
- Bottom Line: *Design a schema that can be explained easily relation by relation. The semantics of attributes should be easy to interpret.*

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
-------	-------	-------	------------	-------	---------	-----	--------	-----------	-----

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
-------	----------------	---------	----------------

DEPT_LOCATIONS

<u>Dnumber</u>	<u>Dlocation</u>
----------------	------------------

PROJECT

Pname	<u>Pnumber</u>	Plocation	Dnum
-------	----------------	-----------	------

WORKS_ON

<u>Essn</u>	<u>Pno</u>	Hours
-------------	------------	-------

DEPENDENT

<u>Essn</u>	<u>Dependent_name</u>	Sex	Bdate	Relationship
-------------	-----------------------	-----	-------	--------------

Figure 5.5

Schema diagram for the COMPANY relational database schema.

Figure 14.1 A simplified COMPANY relational database schema

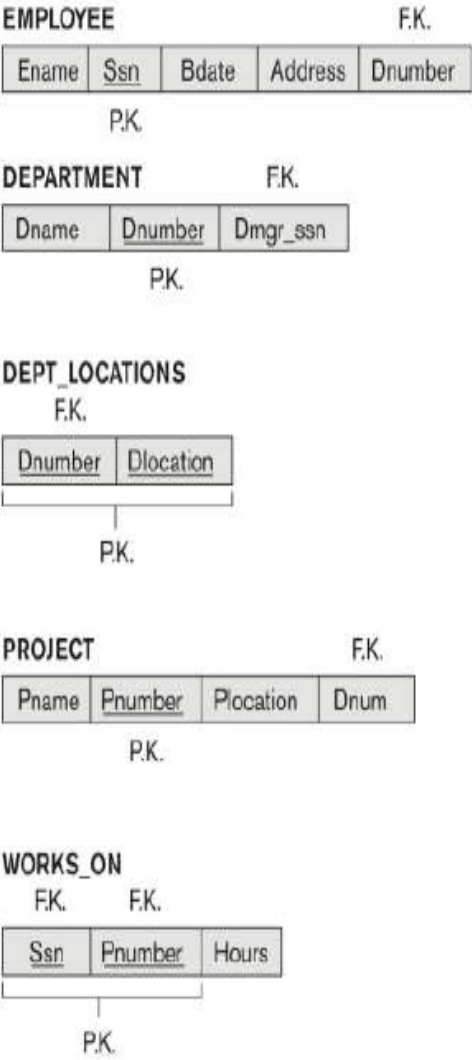


Figure 14.1 A simplified COMPANY relational database schema.

1.2 Redundant Information in Tuples and Update Anomalies

- Information is stored redundantly
 - Wastes storage
 - Causes problems with update anomalies
 - Insertion anomalies
 - Deletion anomalies
 - Modification anomalies

EXAMPLE OF AN INSERT ANOMALY

- Consider the relation:
 - EMP_PROJ(Emp#, Proj#, Ename, Pname, No_hours)
- Insert Anomaly:
 - Cannot insert a project unless an employee is assigned to it.
- Conversely
 - Cannot insert an employee unless an he/she is assigned to a project.

EXAMPLE OF A DELETE ANOMALY

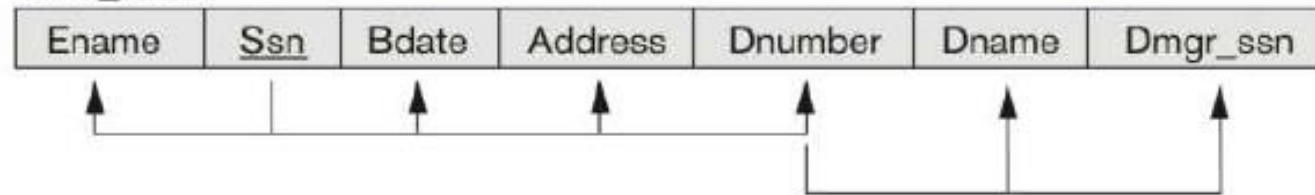
- Consider the relation:
 - EMP_PROJ(Emp#, Proj#, Ename, Pname, No_hours)
- Delete Anomaly:
 - When a project is deleted, it will result in deleting all the employees who work on that project.
 - Alternately, if an employee is the sole employee on a project, deleting that employee would result in deleting the corresponding project.

Figure 14.3 Two relation schemas suffering from update anomalies

Figure 14.3
Two relation schemas
suffering from update
anomalies. (a)
EMP_DEPT and (b)
EMP_PROJ.

(a)

EMP_DEPT



(b)

EMP_PROJ

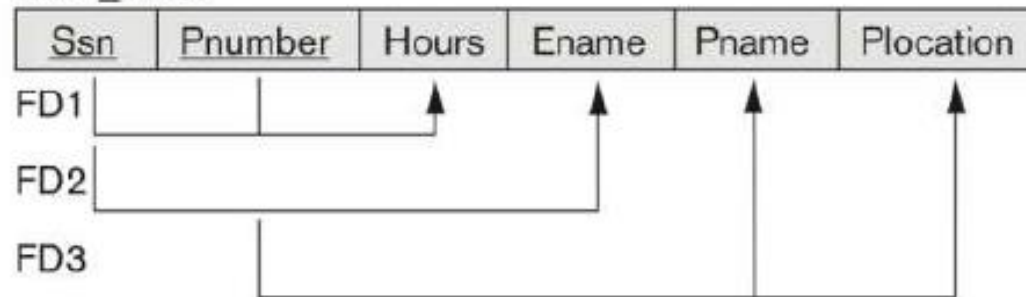


Figure 14.4 Sample states for EMP_DEPT and EMP_PROJ

Figure 14.4

Sample states for EMP_DEPT and EMP_PROJ resulting from applying NATURAL JOIN to the relations in Figure 14.2. These may be stored as base relations for performance reasons.

EMP_DEPT					Redundancy	
Ename	Ssn	Bdate	Address	Dnumber	Dname	Dmgr_ssn
Smith, John B.	123456789	1965-01-09	731 Fondren, Houston, TX	5	Research	333445555
Wong, Franklin T.	333445555	1955-12-08	638 Voss, Houston, TX	5	Research	333445555
Zelaya, Alicia J.	999887777	1968-07-19	3321 Castle, Spring, TX	4	Administration	987654321
Wallace, Jennifer S.	987654321	1941-06-20	291 Berry, Bellaire, TX	4	Administration	987654321
Narayan, Ramesh K.	666884444	1962-09-15	975 FireOak, Humble, TX	5	Research	333445555
English, Joyce A.	453453453	1972-07-31	5631 Rice, Houston, TX	5	Research	333445555
Jabbar, Ahmad V.	987987987	1969-03-29	980 Dallas, Houston, TX	4	Administration	987654321
Borg, James E.	888665555	1937-11-10	450 Stone, Houston, TX	1	Headquarters	888665555

EMP_PROJ			Redundancy		Redundancy	
Ssn	Pnumber	Hours	Ename	Pname	Plocation	
123456789	1	32.5	Smith, John B.	ProductX	Bellaire	
123456789	2	2.5	Smith, John B.	ProductY	Sugarland	
666884444	3	40.0	Narayan, Ramesh K.	ProductZ	Houston	
453453453	1	20.0	English, Joyce A.	ProductX	Bellaire	
453453453	2	20.0	English, Joyce A.	ProductY	Sugarland	
333445555	2	10.0	Wong, Franklin T.	ProductY	Sugarland	
333445555	3	10.0	Wong, Franklin T.	ProductZ	Houston	
333445555	10	10.0	Wong, Franklin T.	Computerization	Stafford	
333445555	20	10.0	Wong, Franklin T.	Reorganization	Houston	
999887777	30	30.0	Zelaya, Alicia J.	Newbenefits	Stafford	
999887777	10	10.0	Zelaya, Alicia J.	Computerization	Stafford	
987987987	10	35.0	Jabbar, Ahmad V.	Computerization	Stafford	
987987987	30	5.0	Jabbar, Ahmad V.	Newbenefits	Stafford	
987654321	30	20.0	Wallace, Jennifer S.	Newbenefits	Stafford	
987654321	20	15.0	Wallace, Jennifer S.	Reorganization	Houston	
888665555	20	Null	Borg, James E.	Reorganization	Houston	

fig14_04.jpg

Guideline for Redundant Information in Tuples and Update Anomalies

- **GUIDELINE 2:**
 - Design a schema that does not suffer from the insertion, deletion and update anomalies.
 - If there are any anomalies present, then note them so that applications can be made to take them into account.

1.3 Null Values in Tuples

- GUIDELINE 3:

- Relations should be designed such that their tuples will have as few NULL values as possible
- Attributes that are NULL frequently could be placed in separate relations (with the primary key)

- Reasons for nulls:

- Attribute not applicable or invalid
- Attribute value unknown (may exist)
- Value known to exist, but unavailable

1.4 Generation of Spurious Tuples – avoid at any cost

- Bad designs for a relational database may result in erroneous results for certain JOIN operations
- The "lossless join" property is used to guarantee meaningful results for join operations
- GUIDELINE 4:
 - The relations should be designed to satisfy the lossless join condition.
 - No spurious tuples should be generated by doing a natural-join of any relations.

Lossless join →
NATURAL JOIN/ JOIN in
MySQL

Spurious Tuples (2)

- There are two important properties of decompositions:
 - a) Non-additive or losslessness of the corresponding join
 - b) Preservation of the functional dependencies.
 - Note that:
 - Property (a) is extremely important and cannot be sacrificed.
 - Property (b) is less stringent and may be sacrificed. (See Chapter 15).
- **spurious tuples** are extraneous or incorrect tuples produced during a join operation, often due to **improper join** conditions or **incorrect table decomposition** or **fail to preserve the integrity** between the tables (i.e. foreign key constraint)
 - With proper database design and careful handling of joins we can avoid spurious tuples.

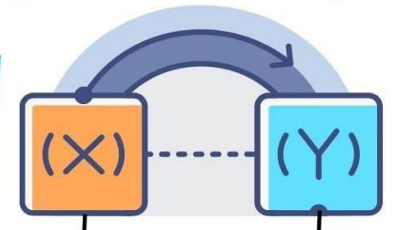
Functional Dependencies

- **Functional dependencies (FDs)** are used to specify *formal measures* of the "goodness" of relational designs.
- **FDs** and keys are used to define **normal forms** for relations.
- **FDs** are **constraints** that are derived from the *meaning* and *interrelationships* of the data attributes .

Functional Dependency in DBMS

Functional dependency in DBMS refers to the relationship between attributes in a database table.

[DatabaseTown.com](https://www.databasetown.com)



EXAMPLE

Armstrong's notation:
{A->B, B->C, C->D}

In a database table of "orders", the attribute "Order ID" functionally determines the attribute "Customer ID". This means that for a given "Order ID", the corresponding "Customer ID" is unique and cannot be changed.

2. Functional Dependencies

- Functional dependencies (FDs)
 - Are used to specify *formal measures* of the "goodness" of relational designs
 - And keys are used to define **normal forms** for relations
 - Are **constraints** that are derived from the *meaning* and *interrelationships* of the data attributes
- A set of attributes X *functionally determines* a set of attributes Y if the value of X determines a unique value for Y

Functional Dependencies (FD)

Mathematical Analogy:

$$f(a) \longrightarrow b$$

e.gs.: $\text{square}(4) \longrightarrow 16$
 $\text{square root}(25) \longrightarrow 5$



Functional Dependencies (FD)

Table (Relation) R

No.	Name
1	Jane
2	Clara
3	Cinderella
4	Haidei
5	Masha

In DBMS,

$A \longrightarrow B$ implies that if we know the value of **A**, we can **determine** the value of **B** from the table R, where **A** and **B** are the attributes of the relation.

In such cases, we say that **A** determines **B**

or

B is determined by **A**

So, $A \longrightarrow B$ is a functional dependency (FD).

In FD, the attribute which is used to determine other attribute is called **Determinant** and the attribute that is determined using the **Determinant** is known as **Dependant**.

Functional Dependencies (FD)

What is FD?

- A **FD** is a constraint that specifies the relationship of one attribute to another attribute in a relation.
- It's important to note that functional dependencies only apply to the **individual tuples** in the table, and not to the table as a whole.

Why **FD**?

- They are used to establish relationships between attributes in a database.
- They are used to ensure that the database is in a ***state of normalization***, which helps to identify and **minimize data redundancy** and **improve data integrity** by identifying and eliminating data inconsistencies in a database.

Functional Dependencies (FD)

How to denote FD?

- A functional dependency is denoted by an arrow “ $\rightarrow$ ”.
- The functional dependency of **A** on **B** is represented by **A** $\rightarrow$ **B**.

Consider a relation with **four attributes A, B, C and D**,

R (ABCD)

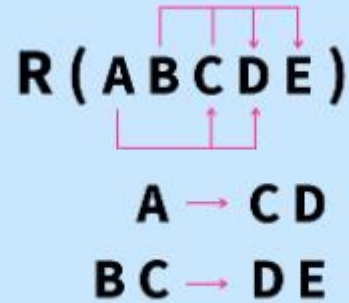
A $\rightarrow$ BCD

B $\rightarrow$ CD

- For the first functional dependency **A $\rightarrow$ BCD**, attributes **B, C and D** are functionally dependent on attribute **A**.
- Function dependency **B $\rightarrow$ CD** has two attributes **C and D** functionally depending upon attribute **B**.

Functional Dependencies (FD)

Functional dependency can also be represented diagrammatically like this,



Pointing arrows determines the depending attribute and the origin of the arrow determines the determinant set.

Functional Dependencies (FD)

Valid and Invalid FDs: case (i)

STUDENT TABLE

Roll_No	Student_Name	Dept_Name	Dept_Building
2	abc	CS	A4
3	pqr	IT	A3
4	xyz	CS	A4
5	xyz	IT	A3
6	mno	EC	B2
7	jkl	ME	B2

FUNCTIONAL DEPENDENCY

ROLL_NO $\longrightarrow$ { STUDENT_NAME, DEPT_NAME
DEPT_BUILDING }

VALID FUNCTIONAL DEPENDENCY

3 $\longrightarrow$ { PQR, IT, A3 }

Determinant	Dependant
1	a
2	b
VALID F.D.	

FD: Valid and Invalid FDs: case (ii)

Roll_No	Student_Name	Dept_Name	Dept_Building
2	abc	CS	A4
3	pqr	IT	A3
4	xyz	CS	A4
5	xyz	IT	A3
6	mno	EC	B2
7	jkl	ME	B2

FUNCTIONAL DEPENDENCY

DEPT_NAME $\longrightarrow$ **DEPT_BUILDING**

VALID FUNCTIONAL DEPENDENCY

making IT **CS** $\longrightarrow$ **A4**



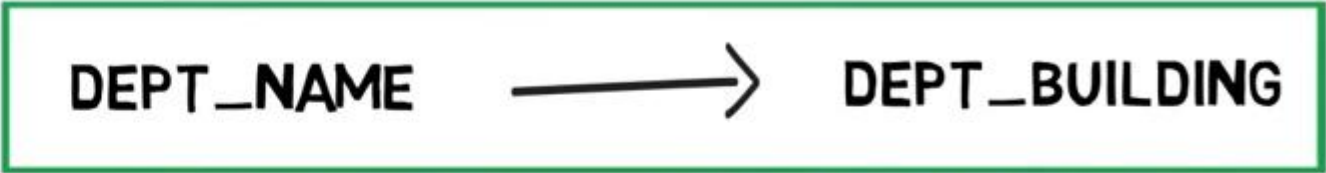
CS $\longrightarrow$ **A4**

Determinant	Dependant
1	a
1	a

VALID F.D.

FD: Valid and Invalid FDs: case (iii)

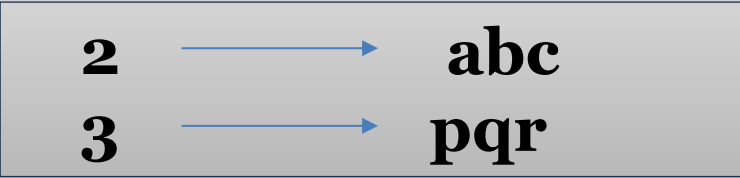
Roll_No	Student_Name	Dept_Name	Dept_Building
2	abc	CS	A4
3	pqr	IT	A3
4	xyz	CS	A4
5	xyz	IT	A3
6	mno	EC	B2
7	jkl	ME	B2



VALID FUNCTIONAL DEPENDENCY



Determinant	Dependant
1	a
2	a
VALID FD	



FD: Valid and Invalid FDs: case (iv)

Roll_No	Student_Name	Dept_Name	Dept_Building
2	abc	CS	A4
3	pqr	IT	A3
4	xyz	CS	A4
5	xyz	IT	A3
6	mno	EC	B2
7	jkl	ME	B2

Determinant	Dependant
1	a
1	b
INVALID F.D.	

FUNCTIONAL DEPENDENCY

STUDENT_NAME ➞ DEPT_NAME

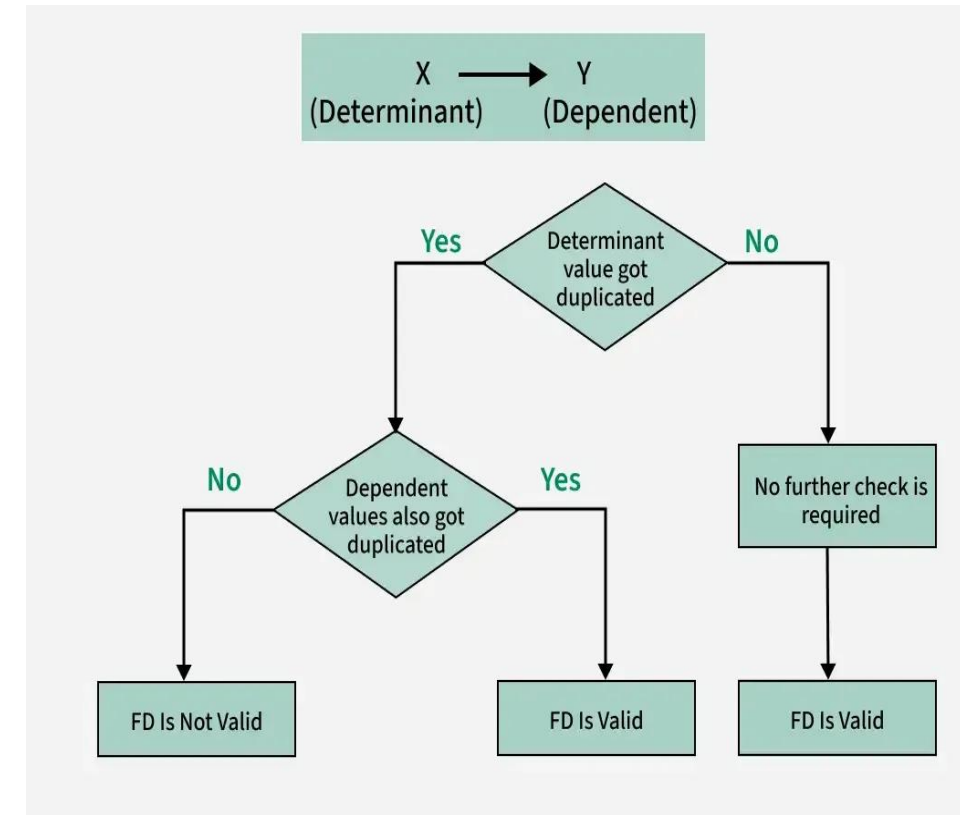
INVALID FUNCTIONAL DEPENDENCY

XYZ ➞ CS

XYZ ➞ IT

2.1 Defining Functional Dependencies

- $X \rightarrow Y$ holds if whenever two tuples have the same value for X , they *must have* the same value for Y
 - For any two tuples $t1$ and $t2$ in any relation instance $r(R)$: If $t1[X]=t2[X]$, *then* $t1[Y]=t2[Y]$
- $X \rightarrow Y$ in R specifies a *constraint* on all relation instances $r(R)$
- Written as $X \rightarrow Y$; can be displayed graphically on a relation schema as in Figures. (denoted by the arrow:).
- FDs are derived from the real-world constraints on the attributes



Examples of FD constraints (1)

- Social security number determines employee name
 - $SSN \rightarrow ENAME$
- Project number determines project name and location
 - $PNUMBER \rightarrow \{PNAME, PLOCATION\}$
- Employee ssn and project number determines the hours per week that the employee works on the project
 - $\{SSN, PNUMBER\} \rightarrow HOURS$

Few more examples of FD

Example 1: employee database

Consider a table in a database that stores employee information:

EmployeeID	EmployeeName	EmployeeAddress	DepartmentID
1	John Doe	123 Main St	D01
2	Jane Smith	456 Elm St	D02
3	Alice Johnson	789 Oak St	D01

EmployeeID $\rightarrow$ **EmployeeName, EmployeeAddress, DepartmentID**: The EmployeeID uniquely determines the EmployeeName, EmployeeAddress, and DepartmentID. Given the EmployeeID, you can find this corresponding information.

Few more examples of FD

Example 2: product database

Consider a product database that stores information about products and their categories:

ProductID	ProductName	CategoryID	CategoryName
101	Laptop	001	Electronics
102	Shirt	002	Clothing
103	Coffee Maker	001	Electronics

ProductID → **ProductName, CategoryID**: The ProductID uniquely determines the ProductName and CategoryID.

CategoryID → **CategoryName**: The CategoryID uniquely determines the CategoryName. For instance, a CategoryID of 001 will always correspond to "Electronics".

Few more examples of FD

Example : bookstore database

Consider a bookstore database that stores information about books and their authors:

ISBN	BookTitle	AuthorID	AuthorName
978-1-56619-909-4	Data Science Handbook	301	John Doe
978-1-56619-909-5	Advanced Python	302	Jane Smith
978-1-56619-909-6	Database Systems	301	John Doe

ISBN $\rightarrow$ BookTitle, AuthorID: The ISBN uniquely determines the BookTitle and AuthorID.

AuthorID $\rightarrow$ AuthorName: The AuthorID uniquely determines the AuthorName.

Examples of FD constraints (2)

- An FD is a property of the attributes in the schema R
- The constraint must hold on *every* relation instance $r(R)$
- If K is a key of R , then K functionally determines all attributes in R
 - (since we never have two distinct tuples with $t1[K]=t2[K]$)

Defining FDs from instances

- Note that in order to define the FDs, we need to understand the meaning of the attributes involved and the relationship between them.
- An FD is a property of the attributes in the schema R
- Given the instance (population) of a relation, all we can conclude is that an FD may exist between certain attributes.
- What we can definitely conclude is – that certain FDs do not exist because there are tuples that show a violation of those dependencies.

Figure 14.7 Ruling Out FDs

Note that given the state of the TEACH relation, we can say that the FD: Text $\rightarrow$ Course may exist. However, the FDs Teacher $\rightarrow$ Course, Teacher $\rightarrow$ Text and Course $\rightarrow$ Text are ruled out.

TEACH

Teacher	Course	Text
Smith	Data Structures	Bartram
Smith	Data Management	Martin
Hall	Compilers	Hoffman
Brown	Data Structures	Horowitz

Figure 14.8 What FDs may exist?

- A relation $R(A, B, C, D)$ with its extension.
- Which FDs may exist in this relation?

A	B	C	D
a1	b1	c1	d1
a1	b2	c2	d2
a2	b2	c2	d3
a3	b3	c4	d3

Armstrong's Axioms/Properties of Functional Dependency in DBMS

- **William Armstrong** in 1974 suggested a few rules related to functional dependency. They are called **RAT rules**.
- **Reflexivity:** If A is a set of attributes and B is a subset of A, then the functional dependency $A \rightarrow B$ holds true.
- For example,
 $A = \{\text{EmployeeID, Name, Department}\}$
 $B = \{\text{EmployeeID, Name}\}$ (subset of A)

Then, $\{\text{EmployeeID, Name, Department}\} \rightarrow \{\text{EmployeeID, Name}\}$

i.e. $\{\text{EmployeeID, Name, Department}\}$ can uniquely determine the $\{\text{EmployeeID, Name}\}$.

Also, $\{\text{Employee_Id, Name}\} \rightarrow \text{Name}$ is valid FD as per reflexivity property.

Armstrong's Axioms/Properties of Functional Dependency in DBMS

- **Augmentation:** If a functional dependency $A \rightarrow B$ holds true, then appending any number of the attribute to both sides of dependency doesn't affect the dependency. It remains true.

For example, $X \rightarrow Y$ holds true then, $ZX \rightarrow ZY$ also holds true.

For example, if $\{ \text{Employee_Id}, \text{Name} \} \rightarrow \{ \text{Name} \}$ holds true then, $\{ \text{Employee_Id}, \text{Name}, \text{Age} \} \rightarrow \{ \text{Name}, \text{Age} \}$

Armstrong's Axioms/Properties of Functional Dependency in DBMS

- **Transitivity:** If two functional dependencies $X \rightarrow Y$ and $Y \rightarrow Z$ hold true, then $X \rightarrow Z$ also holds true by the rule of Transitivity.

For example, if $\{ \text{Employee_Id} \} \rightarrow \{ \text{Name} \}$ holds true and $\{ \text{Name} \} \rightarrow \{ \text{Department} \}$ holds true, then $\{ \text{Employee_Id} \} \rightarrow \{ \text{Department} \}$ also holds true.

Partial Dependencies in DBMS

EXAMPLE

Table R

A	B	C	D
SM	LQ	GT	HY
.	.	.	.
.	.	.	.
.	.	.	.

FUNCTIONAL DEPENDENCIES :-

$A, B \longrightarrow D$

$B \longrightarrow C$

$(A, B) \longrightarrow A, B, C, D$

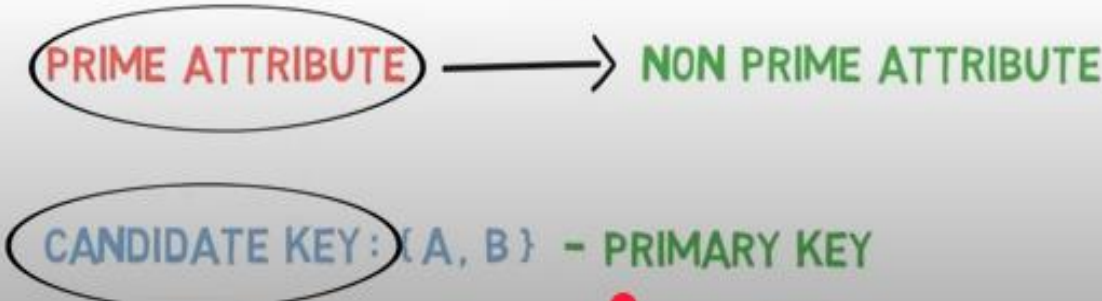
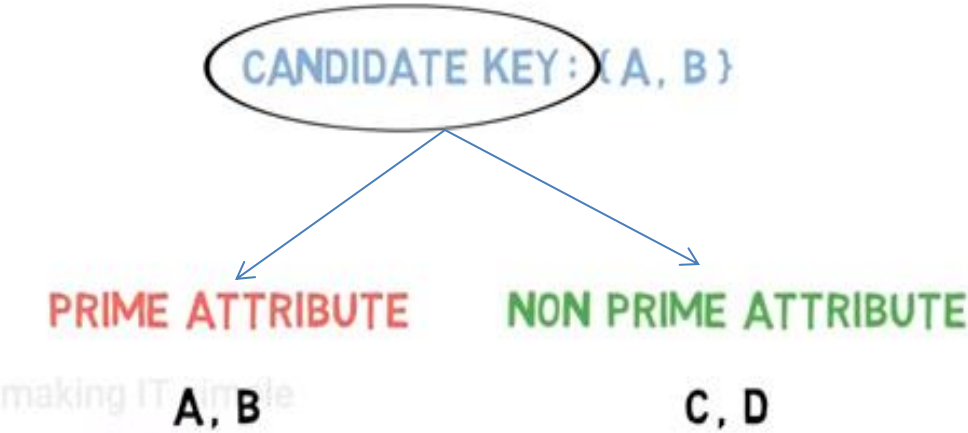
A, B TOGETHER IS A CANDIDATE KEY

$\{ A, B \}^+ = \{ A, B, C, D \} \longrightarrow \text{AB Closure}$

Partial Dependencies in DBMS

Table R

A	B	C	D
SM	LQ	GT	HY
.	.	.	.
.	.	.	.
.	.	.	.



- Attributes that are part of Candidate key are called as PRIME ATTRIBUTES.
- Attributes that are **NOT** part of Candidate key are called as NON PRIME ATTRIBUTES.

Partial Dependencies in DBMS

Table R

A	B	C	D
SM	LQ	GT	HY
.	.	.	.
.	.	.	.
.	.	.	.

CANDIDATE KEY :- {A, B}

B IS NOT A CANDIDATE KEY

PRIME ATTRIBUTE

A, B

NON PRIME ATTRIBUTE

C, D

DETERMINANT {A, B}⁺ = {D} DEPENDANT
COMPLETE CANDIDATE KEY NON PRIME ATTRIBUTE

Attribute D is determined by A, B

DETERMINANT B⁺ = {C} DEPENDANT
PART OF CANDIDATE KEY NON PRIME ATTRIBUTE

PARTIAL DEPENDENCY

A Non prime attribute C can be determined using part of Candidate key i.e. B, this type of dependency is known as **Partial Dependency**.

Partial Dependencies in DBMS: Definition

Partial dependency in a relational database occurs when a non-prime attribute (i.e., not part of any candidate key) is functionally dependent on only a part of the primary key, rather than the entire primary key.

Partial Dependencies in DBMS



DEFINITION

Partial Dependency in DBMS occurs when a non-prime attribute depends on the proper subset or we can say part of the candidate key.

STEPS :-

- 1) IDENTIFY CANDIDATE KEYS
- 2) DIFFERENTIATE PRIME AND NON-PRIME ATTRIBUTES
- 3) FIND A NON-PRIME ATTRIBUTE THAT IS DETERMINED BY A PART OR PROPER SUBSET OF CANDIDATE KEY

Partial Dependencies in DBMS: Various examples to illustrate when does Partial Dependency occur?

1. Consider **Employee_Task** Relation

Employee_ID	Task_No	Employee_Name	Task_Name
C01	34	Mona	App Development
C02	58	Genine	UX/UI Designing

Solution:

We decompose (split) the tables to remove Partial Dependency

Here, Prime (key) attributes are **Employee_ID and Task_No.**

non-prime attributes are **Employee_Name and Task_Name.**

Now, **Employee_Name** can be determined using the **Employee_ID** and hence there exists a **partial dependency.**

Also, **Task_Name** can be determined using the **Task_No.** It makes the relation **Dependent Partially.**

Partial Dependencies in DBMS: Various examples to illustrate when does Partial Dependency occur?

2. Consider **Course_Enrollment** Relation. Can you identify a Partial Dependency if primary key attributes are Student_ID and Course_ID ?

Student_ID	Course_ID	Course_Name	Instructor
1	101	Math	Mr. Smith
1	102	Science	Ms. Johnson
2	101	Math	Mr. Smith
3	103	English	Mr. Brown

Partial Dependencies in DBMS: Various examples to illustrate when does Partial Dependency occur?

3. Consider a table called "Orders" with the following attributes: OrderID, CustomerID, OrderDate, ProductID, and Quantity.

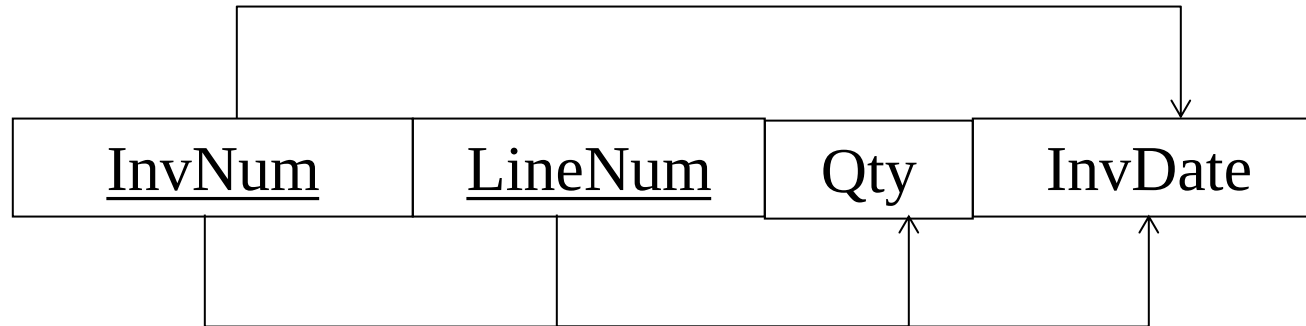
The table's primary key is OrderID, and the non-key attributes are CustomerID, OrderDate, ProductID, and Quantity.

Answer the following:

- (i) If the value of CustomerID depends only on OrderID, check if there is a Partial Dependency?
- (ii) If the combination of OrderID and CustomerID, are prime key attributes, then does there exists a Partial dependency?
- (iii) If the combination of OrderID and ProductID, are prime key attributes, then does there exists a Partial dependency?
- (iv) If the combination of OrderID , CustomerID and ProductID, are prime key attributes, then does there exists a Partial dependency?

Partial Dependencies in DBMS: Various examples to illustrate when does Partial Dependency occur?

e.g.4:



Candidate keys: {InvNum, LineNum} InvDate , which is *partially dependent* on {InvNum, LineNum} as **InvNum is a determinant of InvDate and InvNum is part of a candidate key.**

Types of Functional Dependencies in DBMS

- Trivial functional dependency
- Non-Trivial functional dependency
- Multivalued functional dependency
- Transitive functional dependency

Types of Functional Dependencies in DBMS

Trivial functional dependency

In Trivial Functional Dependency, a dependent is always a subset of the determinant.

i.e. If $X \rightarrow Y$ and Y is the subset of X , then it is called trivial functional dependency.

$X \rightarrow Y$ is called a trivial functional dependency if Y is the subset of X .

A trivial functional dependency occurs when an attribute or a set of attributes determines itself or any subset of itself. It means that, the dependency is obvious and doesn't provide any useful information about the relationship between attributes.

Types of Functional Dependencies in DBMS

Trivial functional dependency

Example 1: $A \rightarrow A$, $B \rightarrow B$

$ABC \rightarrow AB$
 $ABC \rightarrow A$
 $ABC \rightarrow ABC$

Employee_Id	Name	Age
1	Zayn	24
2	Phobe	34
3	Hikki	26
4	David	29

$\{ \text{Employee_Id}, \text{Name} \} \rightarrow \{ \text{Name} \}$ is a Trivial functional dependency, since the dependent **Name** is the subset of determinant $\{ \text{Employee_Id}, \text{Name} \}$.

Types of Functional Dependencies in DBMS

Non-Trivial functional dependency

In Non-trivial functional dependency, the dependent is strictly **not a subset** of the determinant.

i.e. If $X \rightarrow Y$ and Y is not a subset of X , then it is called Non-trivial functional dependency.

Employee_Id	Name	Age
1	Zayn	24
2	Phobe	34
3	Hikki	26
4	David	29

$\{ \text{Employee_Id} \} \rightarrow \{ \text{Name} \}$ is a non-trivial functional dependency because **Name** (dependent) is not a subset of Employee_Id(determinant).

$\{ \text{Employee_Id}, \text{Name} \} \rightarrow \{ \text{Age} \}$ is also a non-trivial functional dependency.

Types of Functional Dependencies in DBMS

Multivalued functional dependency

- In **Multivalued functional dependency**, attributes in the dependent set are **not dependent** on each other.
- For example, $X \twoheadrightarrow \{ Y, Z \}$, if there exists is **no functional dependency** between **Y and Z**, then it is called as *Multivalued functional dependency*.

Types of Functional Dependencies in DBMS

Multivalued functional dependency

Employee_Id	Name	Age
1	Zayn	24
2	Phobe	34
3	Hikki	26
4	David	29
4	Phobe	24

$\{ \text{Employee_Id} \} \rightarrow \{ \text{Name}, \text{Age} \}$ is a Multivalued functional dependency, since the dependent attributes Name, Age are not functionally dependent (i.e. $\text{Name} \rightarrow \text{Age}$ or $\text{Age} \rightarrow \text{Name}$ doesn't exist)

Types of Functional Dependencies in DBMS

Transitive functional dependency

Consider two functional dependencies $\mathbf{A} \rightarrow \mathbf{B}$ and $\mathbf{B} \rightarrow \mathbf{C}$ then according to the transitivity axiom $\mathbf{A} \rightarrow \mathbf{C}$ must also exist. This is called a transitive functional dependency.

i.e. dependent is **indirectly dependent** on determinant in Transitive functional dependency.

Types of Functional Dependencies in DBMS

Transitive functional dependency

Employee_Id	Name	Department	Street Number
1	Zayn	CD	11
2	Phobe	AB	24
3	Hikki	CD	11
4	David	PQ	71
5	Phobe	LM	21

{ Employee_Id $\rightarrow$ Department } and { Department $\rightarrow$ Street Number } holds true.

Hence, according to the axiom of transitivity, { Employee_Id $\rightarrow$ Street Number } is a valid functional dependency.

Trivial Functional Dependencies

A functional dependency is **trivial** if it is satisfied by all instances of a relation.

Example:

$ID, name \rightarrow ID$

$name \rightarrow name$

In general, $\alpha \rightarrow \beta$ is trivial if $\beta \subseteq \alpha$

Takeaway:

The process of Normalization starts with identifying the candidate keys in the relation. Without functional dependency, it's impossible to find candidate keys and normalize the database.

Normal Forms Based on Primary Keys

- Normalization of Relations
- Practical Use of Normal Forms
- Definitions of Keys and Attributes Participating in Keys
- First Normal Form
- Second Normal Form
- Third Normal Form

Normal Forms Based on Primary Keys

Unnormalized form (UNF): A table that contains one or more repeating groups.

Also known as non-first normal form (NF2).

An **Unnormalized Form** describes a database that **doesn't meet** any of the normal form criteria. **NoSQL databases** (MongoDB, Apache etc.) are based on the principles of the unnormalized relational model.

Repeating group: an attribute or group of attributes within a table that occurs with multiple values for a single occurrence of the nominated key attributes of that table.

Customer	
Customer	Phone Number
Jenny	+380955342, +380958362, +3809512535
Loula	+380952373, +380957832

StudentId	Name	Course
101	Raj	Mathematics Chemistry
102	Nilesh	Chemistry
103	Sanu	Physics Chemistry

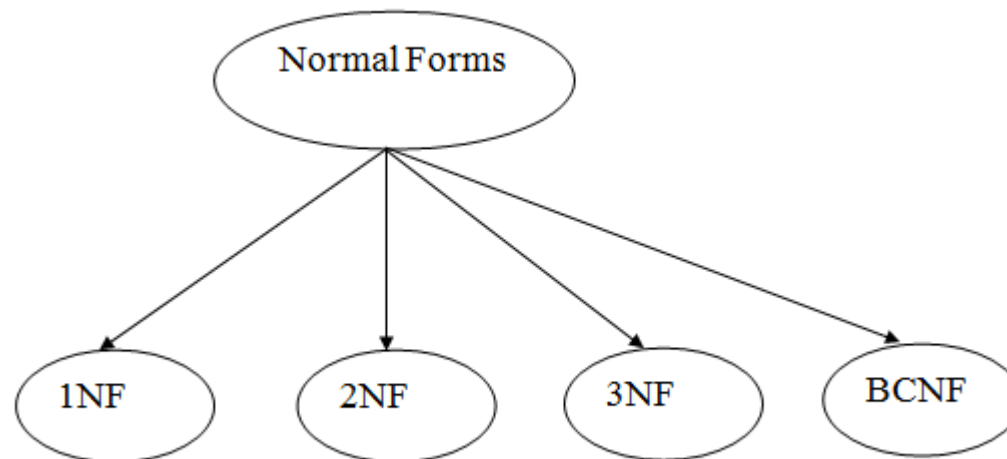
roll_no	name	course
1	Ashish	DBMS
		OS
2	Karan	DBMS

Introduction to Normalization

- Normalization is the process of effectively organizing the data in the database.
- Normalization is a *database design technique* that **reduces data redundancy** from a relation or set of relations and **eliminates** undesirable characteristics like **Insertion, Update and Deletion Anomalies**.
- Normalization divides the larger table into the smaller tables and links them using relationship.
- The purpose of Normalization in SQL is to eliminate redundant (repetitive) data and ensure data is stored logically.

Types of Normal Forms

There are the four types of normal forms



Introduction to Normalization

- **Normalization:** is the process of decomposing unsatisfactory "bad" relations by breaking up their attributes into smaller relations.
- **Normal form:** Condition using keys and FDs of a relation to certify whether a relation schema is in a particular normal form.
 - 2NF, 3NF, BCNF based on keys and FDs of a relation schema
 - 4NF based on keys, multi-valued dependencies
- The inventor of the relational model **Edgar Codd** proposed the theory of **normalization of data** with the introduction of the **First Normal Form**, and he continued to extend theory with Second and Third Normal Form. Later he joined **Raymond F. Boyce** to develop the theory of **Boyce-Codd Normal Form**.

Introduction to Normalization

- The purpose of Normalization in SQL is to eliminate redundant (repetitive) data and ensure data is **stored logically**.
- **Normalization** in relational databases is about logically organizing data to reduce redundancy, ensure consistency, and avoid anomalies (such as update anomalies, delete anomalies). **Physically**, data is still stored on disk in tables, rows, and columns. The database management system (DBMS) will **store the data physically** (on disks or in memory) in a way that optimizes access, indexing, and retrieval (using techniques like **indexing** or **partitioning**). However, **normalization** does not dictate how the DBMS stores the data physically.

Without Normalization (Redundant Data)

OrderID	CustomerName	CustomerAddress	Product	Quantity	OrderDate
1	John Doe	123 Elm St, City X	Laptop	1	2025-03-17
2	John Doe	123 Elm St, City X	Phone	2	2025-03-18
3	Jane Smith	456 Oak Rd, City Y	Tablet	1	2025-03-19
4	John Doe	123 Elm St, City X	Headphones	3	2025-03-20

Customer Table (Stores unique customer information)

CustomerID	CustomerName	CustomerAddress
1	John Doe	123 Elm St, City X
2	Jane Smith	456 Oak Rd, City Y

Order Table (Stores only order-related information with a reference to the customer)

OrderID	CustomerID	Product	Quantity	OrderDate
1	1	Laptop	1	2025-03-17
2	1	Phone	2	2025-03-18
3	2	Tablet	1	2025-03-19
4	1	Headphones	3	2025-03-20

CustomerID is used as a foreign key in the **Order Table**, linking each order to a customer.

Normal Form	Description
<u>1NF</u>	A relation is in 1NF if it contains an atomic value.
<u>2NF</u>	A relation will be in 2NF if it is in 1NF and all non-key attributes are fully functional dependent on the primary key.
<u>3NF</u>	A relation will be in 3NF if it is in 2NF and no transition dependency exists.
<u>4NF</u>	A relation will be in 4NF if it is in Boyce Codd normal form and has no multi-valued dependency.
<u>5NF</u>	A relation is in 5NF if it is in 4NF and not contains any join dependency and joining should be lossless.

The Theory of Data Normalization in MySQL server is still being developed further. For example, there are discussions even on 6th Normal Form. However, in most practical applications, normalization achieves its best in **3rd Normal Form**.

First Normal Form (1NF)

- A Relation will be **1NF** if it contains an **atomic value** (i.e. indivisible or without any group data item) and every relation should possess a **unique primary key** — the smallest set of columns that distinctly identifies a record.
- Disallows composite attributes, multivalued attributes, and **nested relations**; attributes whose values *for an individual tuple* are non-atomic.
- Considered to be part of the definition of relation.
- It states that an attribute of a table ***cannot hold multiple values***. It must hold ***only single-valued attribute***.
- i.e. First normal form disallows the multi-valued attribute, composite attribute, and their combinations.

First Normal Form (1NF)

roll_no	name	course
1	Ashish	DBMS OS
2	Karan	DBMS

This table is **not in 1NF** as it contains the **multivalued attribute**. It can be normalized to the **first normal form** by distributing the multivalued attributes in different columns as follows.

roll_no	name	course_1	course_2
1	Ashish	DBMS	OS
2	Karan	DBMS	

StudentId	Name	Course
101	Raj	Mathematics Chemistry
102	Nilesh	Chemistry
103	Sanu	Physics Chemistry

StudentId	Name	Course1	course2
101	Raj	Mathematics	Chemistry
102	Nilesh	Chemistry	
103	Sanu	Physics	Chemistry

First Normal Form (1NF)

- Lack of unique Primary key
- **Phone Number** attribute holds more than one values.

Customer	
Customer	Phone Number
Jenny	+380955342, +380958362, +3809512535
Loula	+380952373, +380957832

The **Customer ID** column in the **Phone Numbers** table should be designated as a **foreign key**.

Customer	
Customer ID	Customer Name
1	Jenny
2	Loula

Phone Number	
Customer ID	Phone Number
1	+380955342
1	+380958362
1	+380951253
2	+380952373
2	+380957832

Second Normal Form (2NF)

A Relation is in 2 NF if:

- It is in first normal form or 1NF
- It doesn't contain any partial dependencies. i.e. All non-key attributes (i.e., columns not included in the primary key) must be fully dependent on the primary key.

Second Normal Form (2NF)

Employee table with
EmployeeID and DepartmentID as
Candidate keys (Primary keys).

Employee					
EmployeeID	DepartmentID	Name	Surname	Salary	Head
1	3	John	Smith	800	Harry
2	1	Erika	Anderson	1300	Ron
3	2	Bob	Rayan	900	Rick
4	3	Oliver	Williams	950	Harry
2	2	Erika	Anderson	880	Rick

Further, there is a Modification anomaly, if we try to delete *DepartmentID* row from the table, we can't determine *Head* in such case.

Check if Employee table is in 2NF:

1. Table is in 1 NF as there is a composite Primary key. Also, Table has atomic values.
2. The non-prime attributes, Name, Surname, and Head are dependent only on part of the composite Primary key. So **violates** the rule, and hence the table is not in 2NF.

Second Normal Form (2NF)

Employee		
EmployeeID	Name	Surname
1	John	Smith
2	Erika	Anderson
3	Bob	Rayan
4	Oliver	Williams

Department	
DepartmentID	Head
1	Ron
2	Rick
3	Harry

Second Normal Form (2NF)

Salary		
EmployeeID	DepartmentID	Salary
1	3	800
2	1	1300
3	2	900
4	3	950
2	2	880

Now, the table aligns with the **second normal form**, guaranteeing we **retain vital information** even if rows from the initial table are deleted.

Second Normal Form (2NF)

A table that is in 2NF (Second Normal Form) can still suffer from **anomalies**, specifically **update anomalies, insert anomalies, and delete anomalies**. These issues are typically addressed by further normalizing the table to **3NF (Third Normal Form)** or beyond. 2NF doesn't guarantee to resolve the following issues:

- **Transitive Dependencies:** These occur when a non-key column (i.e. non-prime attribute) is dependent on another non-key column, which is then dependent on the primary key. This is the main issue addressed by 3NF.
- **Anomalies:** Even in 2NF, if there are transitive dependencies, the following anomalies may be faced: **update anomalies, insert anomalies, and delete anomalies**.

Second Normal Form (2NF)

Example of a Table in 2NF with anomalies. Consider the table STUDENT, stores information about students and the teachers.

Stu_Id	Stu_Name	Teach_Id	Teach_Name	Teach_Qual
2523	Anurag	201	Mohan	MCA
3712	Raju	202	Ravi	M.Tech
4906	Raman	203	Mahima	Ph.D
2716	Jyoti	204	Anjali	MCA
1768	Meetali	205	Sonia	M.Tech

- ✓ Here, the primary key is Stu_Id.
- ✓ The table is in 2 NF.
- However, this table suffers from insertion, deletion and update anomalies.

Second Normal Form (2NF)

Insertion anomaly: Suppose that we want to insert information about a new teacher 'Mayank' having Teach_Id = '206' , Teach_Qual = 'MCA' who has not yet been allotted any student. This is not possible as Stu_Id will contain a *null value*.

Stu_Id	Stu_Name	Teach_Id	Teach_Name	Teach_Qual
2523	Anurag	201	Mohan	MCA
3712	Raju	202	Ravi	M.Tech
4906	Raman	203	Mahima	Ph.D
2716	Jyoti	204	Anjali	MCA
1768	Meetali	205	Sonia	M.Tech
NULL	NULL	206	Mayank	MCA

The table assumes that every record in the table represents a student-teacher relationship (i.e., each student is associated with a teacher).

Second Normal Form (2NF)

Deletion anomaly: Suppose that a student whose Stu_Id = 1768 decides to quit the college, so we would have to delete this tuple from the STUDENT relation. Thus, on deleting this tuple, the information about the teacher would also be deleted.

Updation Anomaly: The second normal form also suffers from updation anomaly.

e.g. If the value of the qualifications of the teacher i.e. Teach_Qual whose Teach_Id = '204' is updated from MCA to Ph.D. , then we would have to update this information every row wherever this appears, which may lead to inconsistencies in the database.

How to overcome the anomalies?

Solution: Normalize the table further into **3NF**.

Review

1NF eliminated Composite and Multivalued Attributes

2NF eliminated Partial Dependency

3NF.....



Third Normal Form (3NF)

A Relation is in 3 NF if:

- It is in second normal form or 2NF.
- It doesn't contain transitive functional dependencies (for non-prime attributes).

Transitive dependency means indirect dependency

i.e. $\mathbf{X \rightarrow Y}$ (Y is dependent on X) and

$\mathbf{Y \rightarrow Z}$

so, $\mathbf{X \rightarrow Z}$.

3 NF is used to reduce the **data duplication**. It is also used to achieve the data integrity.

Third Normal Form (3NF)

e.g.:

Emp_Id	Emp_Name	Mob_No
1	ABC	7485961232
2	DEF	8596742123
3	GHI	9674851323

FUNCTIONAL DEPENDENCIES :

EMP_ID  **EMP_NAME, MOB_NO**

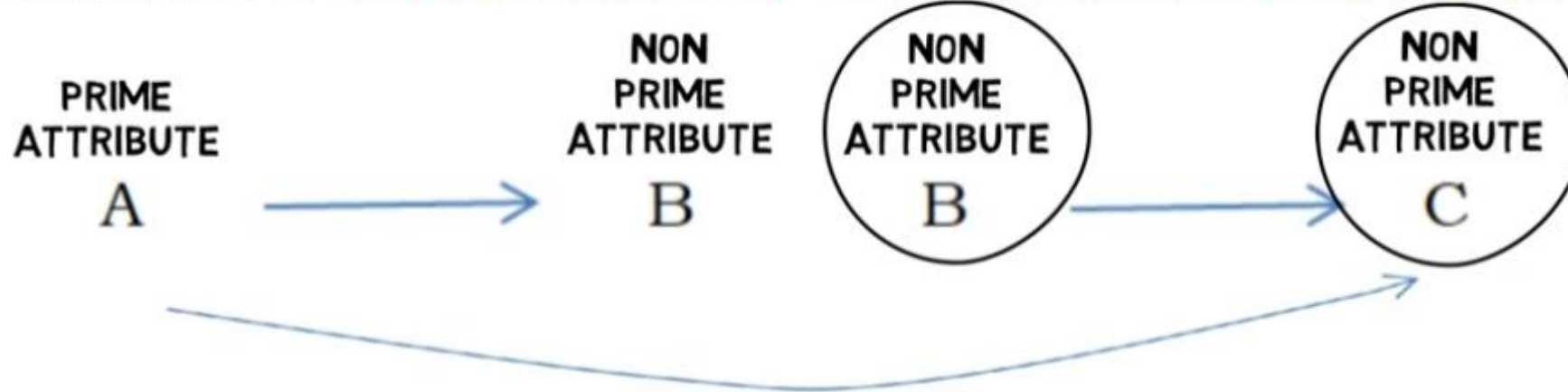
1st condition:

- ✓ No Composite and Multivalued Attributes  1NF
- ✓ No Partial Dependency  2 NF

Third Normal Form (3NF)

2nd condition: Table must not contain Transitive Dependency.

CONDITION 2: TABLE MUST NOT CONTAIN TRANSITIVE DEPENDENCY



Third Normal Form (3NF)

Consider the table BOOK. BookID is the primary key of the table. Check if the table is in 3NF?

BOOK

BookID	CategoryID	Category Type	Price
1	1	History	300
2	2	Science	350
3	1	History	250
4	3	Mystery	200
5	2	Science	300

FDs:

BookID → CategoryID

CategoryID → Category Type

So,

BookID → Categorytype

Hence, the table has Transitive Dependency. So, table is not in 3NF.

Further, there is **redundancy** in the above table, which may be removed with the 3NF.

Third Normal Form (3NF)

How to make the table BOOK to be in 3 NF?

Solution: Decompose the table into multiple tables.

BOOK₁

BookID	CategoryID	Price
1	1	300
2	2	350
3	1	250
4	3	200
5	2	300

- ✓ Nonprime attributes, CategoryID and Price are fully functionally dependent on BookID. So, table is in 2NF, i.e. the 1st condition for 3NF is satisfied.
- ✓ No Transitive FD in this table is found.
- ✓ Hence, this table BOOK is in 3NF.

Third Normal Form (3NF)

How to make the table BOOK to be in 3 NF?

Solution: Decompose the table into multiple tables.

CATEGORY

CategoryID	Category Type
1	History
2	Science
3	Mystery

Primary key Nonprime attribute

- ✓ Non-prime attribute, CategoryType is fully functionally dependent on CategoryID, which is the primary key. Hence table is in 2NF.
- ✓ No Transitive FD in this table is found.
- ✓ Hence, table CATEGORY is in 3NF.

Note that Redundant data has been eliminated in the above table.

Third Normal Form (3NF)

e.g.2:



PRIMARY KEY

Emp_ID	Emp_Name	Project_ID	Project_Name
1	ABC	123	X
2	DEF	789	Z
3	GHI	123	X
4	JKL	123	X
5	MNO	789	Z
6	PQR	789	Z
7	STU	123	X

FUNCTIONAL DEPENDENCIES :

EMP_ID → EMP_NAME, PROJECT_ID

PROJECT_ID → PROJECT_NAME

Emp ID indirectly determines the Project_Name. A non-prime attribute Project_ID determines another non-prime attribute, Project_Name. So, in this table **Transitive Dependency** exists and hence the table is **not in 3NF**.

Third Normal Form (3NF)

How to eliminate Transitive Dependency so that the table exists in 3NF.

Third Normal Form (3NF)

Emp_ID	Emp_Name	Project_ID	Project_Name
1	ABC	123	X
2	DEF	789	Z
3	GHI	123	X
4	JKL	123	X
5	MNO	789	Z
6	PQR	789	Z
7	STU	123	X

THIS TABLE IS IN 2NF

making IT simple



WHY 3NF ???

**IN HUGE DATABASE THIS WILL
LEAD TO WASTE OF STORAGE**



Primary Key		Foreign Key
Emp_ID	Emp_Name	Project_ID
1	ABC	123
2	DEF	789
3	GHI	123
4	JKL	123
5	MNO	789
6	PQR	789
7	STU	123

Functional Dependencies:

Emp_ID $\longrightarrow$ Emp_Name, Project_ID

Primary Key

Project_ID	Project_Name
123	X
789	Z

Functional Dependencies:

Project_ID $\longrightarrow$ Project_Name



Boyce-Codd Normal Form (BCNF) or 3.5 NF

- BCNF is the advance version of 3NF, hence known as 3.5 NF.
- It is stricter than 3NF.
- Every relation in BCNF is also in 3NF; however, a relation in 3NF is not necessarily in BCNF.

BCNF Rule: A table is in BCNF if

- The table is in 3NF.
- **Every functional dependency** $X \rightarrow Y$, X is the super key of the table.

Boyce-Codd Normal Form (BCNF) or 3.5 NF

- **Super key:** a **super key** is a set of one or more attributes (columns) that can uniquely identify a record (or tuple) in a database table.
- A superkey **may or may not** be minimal, meaning it can have extra attributes that aren't necessary for uniqueness.
- **superkey cannot** have **duplicate values** in the context of a database table.
- A primary key is a **candidate key** that has been **chosen** to uniquely identify records in a table. A primary key must be **minimal**, meaning it cannot have any unnecessary attributes.

Boyce-Codd Normal Form (BCNF) or 3.5 NF

Example with Duplicate Values:

Consider this table:

EmployeeID	Name	Email
101	John	john@email.com
102	Alice	alice@email.com
102	Bob	bob@email.com

{EmployeeID} can not be a valid superkey.

Instead, the combination {EmployeeID, Email} would work as a **valid superkey** because the pair of values will **uniquely identify each record**.

Boyce-Codd Normal Form (BCNF) or 3.5 NF

e.g. 1:

CONDITION 1: TABLE MUST BE IN THIRD NORMAL FORM (3NF)

Student	Subject	Teacher
ABC	Database	MNO
ABC	C	PQR
XYZ	Database	MNO
XYZ	C	STU

FUNCTIONAL DEPENDENCIES :

Student, Subject $\longrightarrow$ Teacher

Teacher $\longrightarrow$ Subject

CANDIDATE KEY: { STUDENT, SUBJECT }

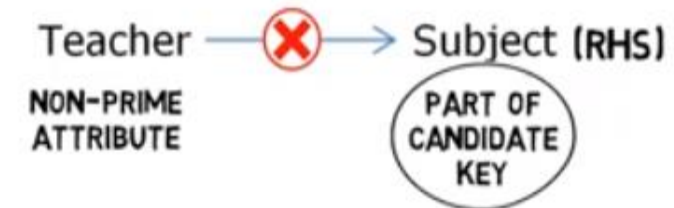
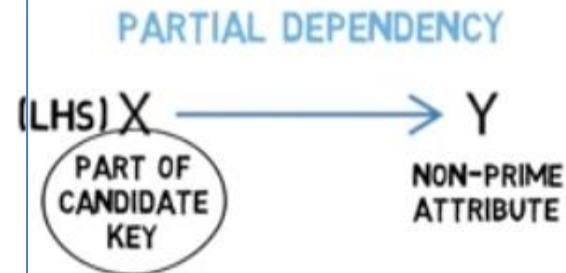
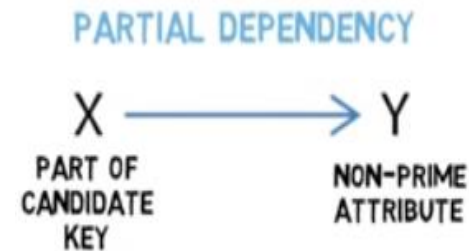
Super key:

Making IT simple



Boyce-Codd Normal Form (BCNF) or 3.5 NF

1. Check if the table is in **1NF**. ✓
2. Check if the table is in **2NF**. ✓



Boyce-Codd Normal Form (BCNF) or 3.5 NF

3. Check if the table is in **3NF**. ✓

TRANSITIVE DEPENDENCY



TRANSITIVE DEPENDENCY



Boyce-Codd Normal Form (BCNF) or 3.5 NF

CONDITION 2: EVERY FUNCTIONAL DEPENDENCY IN THE TABLE IS OF FORM $X \rightarrow Y$ WHERE X OR LHS IS STRICTLY A CANDIDATE KEY OR A SUPER KEY.

Student	Subject	Teacher
ABC	Database	MNO
ABC	C	PQR
XYZ	Database	MNO
XYZ	C	STU

Candidate Key: {Student, Subject}

Functional Dependencies:

Student, Subject $\rightarrow$ Teacher

Teacher $\rightarrow$ Subject

(LHS) $X \rightarrow Y$
CANDIDATE
OR A SUPER
KEY

Student, Subject $\xrightarrow{\checkmark}$ Teacher
CANDIDATE
KEY NON-PRIME
ATTRIBUTE



Boyce-Codd Normal Form (BCNF) or 3.5 NF

CONDITION 2: EVERY FUNCTIONAL DEPENDENCY IN THE TABLE IS OF FORM $X \rightarrow Y$ WHERE X OR LHS IS STRICTLY A CANDIDATE KEY OR A SUPER KEY.

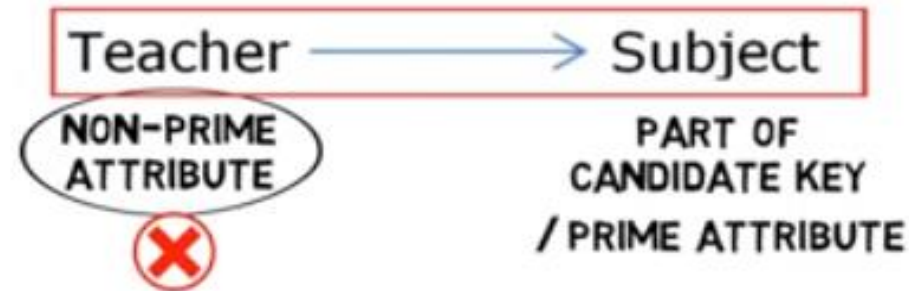
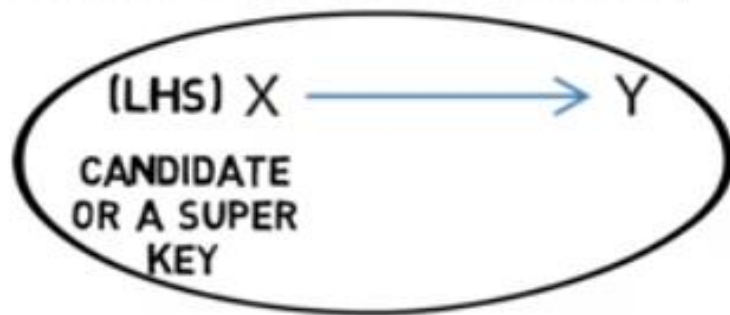
Student	Subject	Teacher
ABC	Database	MNO
ABC		PQR
XYZ	Database	MNO
XYZ	C	STU

Functional Dependencies:

Student, Subject $\rightarrow$ Teacher

Teacher $\rightarrow$ Subject

Candidate Key: {Student, Subject}



Hence, table is not in BCNF or 3.5 NF

Boyce-Codd Normal Form (BCNF) or 3.5 NF

How to make the table to be in BCNF: **Decompose the table into 2 tables , R1 and R2 as shown below.**

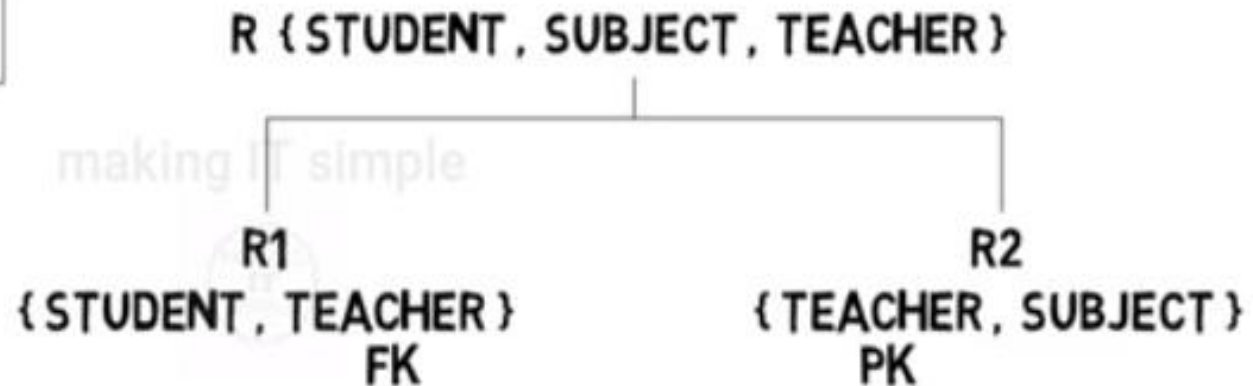
Student	Subject	Teacher
ABC	Database	MNO
ABC	C	PQR
XYZ	Database	MNO
XYZ	C	STU

Candidate Key: {Student, Subject}

Functional Dependencies:

Student, Subject $\longrightarrow$ Teacher

Teacher $\longrightarrow$ Subject



BOYCE - CODD NORMAL FORM

Boyce-Codd Normal Form (BCNF) or 3.5 NF

R1

STUDENT	TEACHER
ABC	MNO
ABC	PQR
XYZ	MNO
XYZ	STU

R2

TEACHER	SUBJECT
MNO	DATABASE
PQR	C
STU	C

R1 and R2 are in BCNF.

Introduction to the higher normal forms

Fourth Normal Form (4NF)

- Unlike 2NF, 3NF, BCNF which are concerned with functional dependencies, 4NF is concerned with **multi valued dependency (MVD)**.

A relation will be in 4NF if it is in Boyce Codd normal form (BCNF) and has no multi-valued dependency.

For a dependency $A \twoheadrightarrow B$, if for a single value of A, multiple values of B exists, then the relation will be a multi-valued dependency.

Introduction to the higher normal forms

MVD (Multi-Valued Dependency) is a concept in **database theory** related to the relationships between attributes in a **relational database**. It is an extension of **functional dependency** and is important when dealing with **higher normal forms**, specifically **Fourth Normal Form (4NF)**.

Definition of MVD:

A **multi-valued dependency (MVD)** occurs when one attribute (or a set of attributes) determines a set of other attributes in a way that the values of the determined attributes are independent of the values of other attributes in the relation.

From the definition of multivalued dependency, we can derive the following rule:

- If $\alpha \rightarrow \beta$, then $\alpha \twoheadrightarrow \beta$

That is, every functional dependency is also a multivalued dependency.

MVD is denoted as **$A \twoheadrightarrow B$** , in which attribute B is determined by a single value of A.

Introduction to the higher normal forms

Note: In DBMS, 1NF (First Normal Form) focuses on eliminating **non-atomic** values (*like storing multiple values in a single cell*), while **multivalued dependencies (MVDs)** address situations where *one attribute can have multiple, independent values*, which is a concern addressed in higher normal forms like 4NF.

Student ID	Name	Phone Numbers
101	Alice	(123) 456-7890, (987) 654-3210
102	Bob	(555) 123-4567



Non-atomic values in 1NF.

Employee ID	Skill	Project
101	Java	Project1
101	Python	Project2
102	C++	Project1
102	Java	Project3



Multivalued Dependency.

Introduction to the higher normal forms

Example 1: The concept of MVD or Multivalued dependency refers to having multiple rows in a given table.

There is a car manufacturing company that produces two of the colours in the market, i.e., red and yellow of each of their models, every year.

CAR_MODEL	MANUF_MONTH	COLOUR
S2011	JAN	Yellow
S2001	FEB	Red
S3001	MAR	Yellow
S3001	APR	Red
S4006	MAY	Yellow
S4006	JUN	Red

The columns COLOUR and MANUF_MONTH are dependent on CAR_MODEL, and they are independent of each other. Thus, we can call both of these columns multivalued. These are, as a result, dependent on CAR_MODEL.

The dependencies are:

$CAR_MODEL \twoheadrightarrow MANUF_MONTH$

$CAR_MODEL \twoheadrightarrow COLOUR$

i.e. “CAR_MODEL multidetermined MANUF_MONTH” and “CAR_MODEL multidetermined COLOUR”.

Introduction to the higher normal forms

Fourth Normal Form (4NF)

- Unlike 2NF, 3NF, BCNF which are concerned with functional dependencies, 4NF is concerned with **multi valued dependency (MVD)**.

A relation will be in 4NF if it is in Boyce Codd normal form (BCNF) and has no multi-valued dependency.

For a dependency $A \twoheadrightarrow B$, if for a single value of A, multiple values of B exists, then the relation will be a multi-valued dependency.

Introduction to the higher normal forms : (4NF)

Course	Instructor	Textbook
DBMS	ABC	NAVATHE RAGHU
	XYZ	
	PQR	GREG
OS	MNO	PETER

Course $\rightarrow\rightarrow$ Instructor

Course $\rightarrow\rightarrow$ Textbook

However, Instructor and Textbook are not related.

MVD exists in this table and hence table can't be in 4 NF. How to make it to be in 4 NF?

Decompose tables into 2 tables as shown below.

TEACHER

Course	Instructor
DBMS	ABC
DBMS	XYZ
DBMS	PQR
OS	MNO

TEXTBOOK

Course	Textbook
DBMS	NAVATHE
DBMS	RAGHU
OS	GREG
OS	PETER

Introduction to the higher normal forms

Fifth Normal Form (5NF)

A relation is in **5NF** if it is in 4NF and not contains any **join dependency** and joining should be **lossless**.

5NF is satisfied when all the tables are broken into as many tables as possible in order to avoid redundancy.

5NF is also known as Project-Join Normal Form (PJ/NF).

Introduction to the higher normal forms: 5NF

Concept of 5th Normal Form

- We can understand the 5th NF by understanding either join dependency or breaking down the tables into parts and rejoin.
- As Join dependency is a little bit confusing topic so let understand the breaking down the tables into parts.

Suppose a table SPC with composite primary key {Supplier, product, customer}

Suplier	Product	Customer
Ali	ABC	Nauman
Ahmad	KLM	Qasim
Haider	XYZ	Saim

Table: SPC

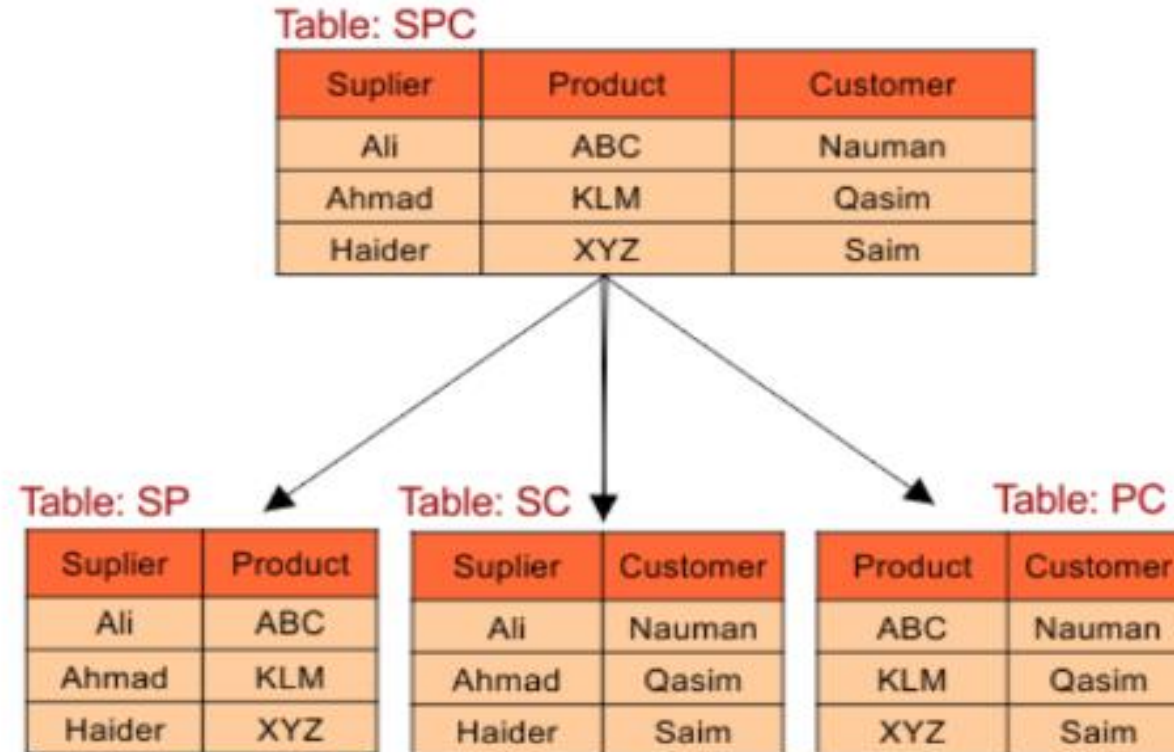
In the above table, supplier supplies products and customer can use these products but note that supplier does not directly supply to any customer.

In simple word, Supplier ("Ali") produce ("ABC") and Customer ("Nauman") can use it. But Ali and Nauman are not directly connected.

Introduction to the higher normal forms: 5NF

Decomposition

After Decomposition of table (SPC) in to three parts as given below



Note that, In table (SC) Supplier and Customer are directly connected. So values are changed after decomposition of table (SPC) into parts.

So above table is in 5NF.

Introduction to the higher normal forms

Sixth Normal Form (6NF)

A relation is in 6NF, only if, It is in 5NF, and every join dependency on the relation is trivial.

6NF is primarily used to deal with **temporal databases**, where the database includes information that changes over time, such as historical data.

REVIEW:

Following are the various types of Normal forms:

	1NF	2NF	3NF	4NF	5NF
Decomposition of Relation	R	R ₁₁ R ₁₂	R ₂₁ R ₂₂ R ₂₃	R ₃₁ R ₃₂ R ₃₃ R ₃₄	R ₄₁ R ₄₂ R ₄₃ R ₄₄ R ₄₅
Conditions	Eliminate Repeating Groups	Eliminate Partial Functional Dependency	Eliminate Transitive Dependency	Eliminate Multi-values Dependency	Eliminate Join Dependency

REVIEW:

Normal Form	Description
1NF	A relation is in 1NF if it contains an atomic value.
2NF	A relation will be in 2NF if it is in 1NF and all non-key attributes are fully functional dependent on the primary key.
3NF	A relation will be in 3NF if it is in 2NF and no transition dependency exists.
BCNF	A stronger definition of 3NF is known as Boyce Codd's normal form.
4NF	A relation will be in 4NF if it is in Boyce Codd's normal form and has no multi-valued dependency.
5NF	A relation is in 5NF. If it is in 4NF and does not contain any join dependency, joining should be lossless.

Practice questions

1. For a database relation $R(a,b,c,d)$, where the domains, a, b, c, d include only atomic values, only the following functional dependencies and those that can be inferred from them hold
 - $a \longrightarrow c$
 - $b \longrightarrow d$

This relation is

- (i) in first normal form but not in second normal form
- (ii) in second normal form but not in first normal form
- (iii) in third normal form
- (iv) none of the above

Practice questions

2.

⬆ Consider the following relational schema along with all the functional dependencies that hold on them.

0

⬇

$R1(A, B, C, D, E): \{D \rightarrow E, EA \rightarrow B, EB \rightarrow C\}$

$R2(A, B, C, D): \{A \rightarrow D, A \rightarrow B, C \rightarrow A\}$

Which of the following statement(s) is/are TRUE?

- A. $R1$ is in 3NF
- B. $R2$ is in 3NF
- C. $R1$ is NOT in 3NF
- D. $R2$ is NOT in 3NF

Consider the following relational schema along with all the functional dependencies that hold on them.

$R1(A, B, C, D, E) : \{D \rightarrow E, EA \rightarrow B, EB \rightarrow C\}$

$R2(A, B, C, D) : \{A \rightarrow D, A \rightarrow B, C \rightarrow A\}$

Which of the following statement(s) is/are TRUE?

$R1$ is in 3NF

$R2$ is in 3NF

$R1$ is NOT in 3NF

$R2$ is NOT in 3NF

3.



Consider a relational schema $team(name, city, owner)$, with functional dependencies $\{name \rightarrow city, name \rightarrow owner\}$.

0



The relation $team$ is decomposed into two relations, $t1(name, city)$ and $t2(name, owner)$. Which of the following statement(s) is/are TRUE?

- A. The relation $team$ is NOT in BCNF .
- B. The relations $t1$ and $t2$ are in BCNF.
- C. The decomposition constitutes a lossless join.
- D. The relation $team$ is NOT in 3 NF .

4.



Consider the following statements in the context of relational database models.

23

A: If a relation R is in $3NF$ but not in BCNF then R has a nontrivial functional dependency $X \rightarrow Y$, where X is a proper subset of a candidate key and Y is a proper subset of a different candidate key.



B: For all sets of functional dependencies F and G , $F^+ \cup G^+ = (F \cup G)^+$.

Which of the above statements is/are TRUE?

- A. Only A
- B. Only B
- C. Both A and B
- D. None

5.

Relation R has eight attributes $ABCDEFGH$. Fields of R contain only atomic values.

$F = \{CH \rightarrow G, A \rightarrow BC, B \rightarrow CFH, E \rightarrow A, F \rightarrow EG\}$ is a set of functional dependencies (FDs) so that F^+ is exactly the set of FDs that hold for R .

The relation R is

- A. in $1NF$, but not in $2NF$.
- B. in $2NF$, but not in $3NF$.
- C. in $3NF$, but not in $BCNF$.
- D. in $BCNF$

6.

Given the relational schema $R = (U, V, W, X, Y, Z)$ and the set of functional dependencies:

$$\{U \rightarrow V, U \rightarrow W, WX \rightarrow Y, WX \rightarrow Z, V \rightarrow X\}$$

Which of the following functional dependencies can be derived from the above set?

- A. $VW \rightarrow YZ$
- B. $WX \rightarrow YZ$
- C. $VW \rightarrow U$
- D. $VW \rightarrow Y$

7.

Consider the following database relation: $R = (SFLNGCRWH) = (S : \text{SID}, F : \text{First Name}, L : \text{Last Name}, N : \text{Number of Units}, G : \text{GPA}, C : \text{Course (of teaching)}, R : \text{Rating for class}, W : \text{Wage}, H : \text{Hours per week of work})$

Given the following set of functional dependencies (FDs):

$F = \{S \rightarrow FL, S \rightarrow NG, SC \rightarrow R, CH \rightarrow W, FL \rightarrow R, C \rightarrow H, H \rightarrow C\}$,

which of the following additional FDs (added to the set in isolation from the other answer choices) would reduce the size of the candidate key if it were the only one applied?

- A. SID determines Rating
- B. Rating determines Wage
- C. Rating determines Course
- D. Rating and First Name determine Wage and Hours of Work

8.

A database of research articles in a journal uses the following schema.

(VOLUME, NUMBER, STARTPAGE, ENDPAGE, TITLE, YEAR, PRICE)

The primary key is (**VOLUME, NUMBER, STARTPAGE, ENDPAGE**) and the following functional dependencies exist in the schema.

(VOLUME, NUMBER, STARTPAGE, ENDPAGE) → TITLE

(VOLUME, NUMBER) → YEAR

(VOLUME, NUMBER, STARTPAGE, ENDPAGE) → PRICE

The database is redesigned to use the following schemas.

(VOLUME, NUMBER, STARTPAGE, ENDPAGE, TITLE, PRICE)

(VOLUME, NUMBER, YEAR)

Which is the weakest normal form that the new database satisfies, but the old one does not?

A 1NF

B 2NF

Correct Answer

C 3NF

D BCNF

9.

Which normal form is considered adequate for normal relational database design?

A $2NF$

B $5NF$

C $4NF$

D $3NF$

Correct Answer

Practice questions

10.

GATE-2001

2. Consider a schema $R(A, B, C, D)$ and functional dependencies $A \rightarrow B$ and $C \rightarrow D$. Then the decomposition of R into $R_1(A, B)$ and $R_2(C, D)$ is

- (a) dependency preserving and lossless join
- (b) lossless join but not dependency preserving
- (c) dependency preserving but not lossless join
- (d) not dependency preserving and not lossless join

Ans: option (c)

Explanation:

While decomposing a relational table we must verify the following properties:

i) Dependency Preserving Property: A decomposition is said to be dependency preserving if $F^+ = (F_1 \cup F_2 \cup \dots \cup F_n)^+$, Where F^+ = total functional dependencies (FDs) on universal relation R , F_1 = set of FDs of R_1 , and F_2 = set of FDs of R_2 .

For the above question R_1 preserves $A \rightarrow B$ and R_2 preserves $C \rightarrow D$. Since the FDs of universal relation R is preserved by R_1 and R_2 , the decomposition is dependency preserving.

THANK YOU